

Software requirements:

- Google Android Things
- Android Studio
- Etcher image burner

The experiment: So let's set up the Android Things console. Go to <https://www.partner.android.com/things/console/> for this. To create a new product, fill in the necessary details after selecting *Create Product* (Figure 2).

Now, create a new model or choose an already existing model. The model name is unique in that it includes letters and numbers.

Select *Build Menu*. A new build can be created from scratch, using *Starter Kit*, or can be created from an existing build, which may not exist during the initial setup (Figure 3).

Name the build. Select the latest available model. In our case, it is OS Model 1.0.2.

There are options to select applications that need to be pre-installed. Build resources can be added using the next two options.

Hardware configurations and partitions can be changed if required. The total size can be also provided in the last menu (Figure 4).

Creating the build configuration can take a few seconds. Next, download the development package, which will be a .zip file, containing the boot image of Android Things configured.

Connecting to Raspberry Pi

The next step is to burn the image to an SD card for booting into Raspberry Pi. Etcher is a good option for the task. Use it to burn the image extracted from the .zip file that has been downloaded.

When Raspberry Pi is turned on with the inserted SD card, Android Things loads up. Next, connect to a Wi-Fi network (Figure 5).

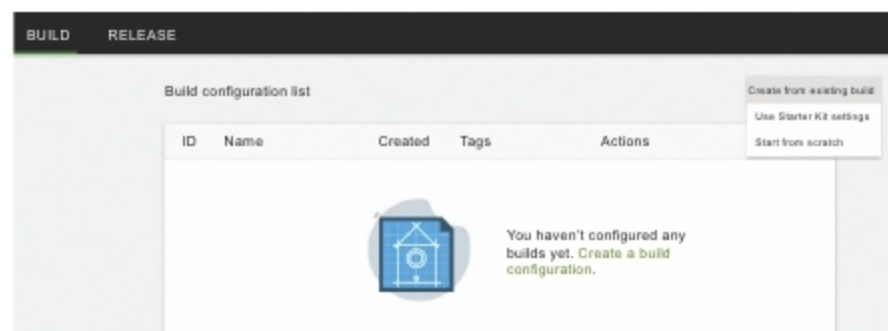


Figure 3: Creating a new build

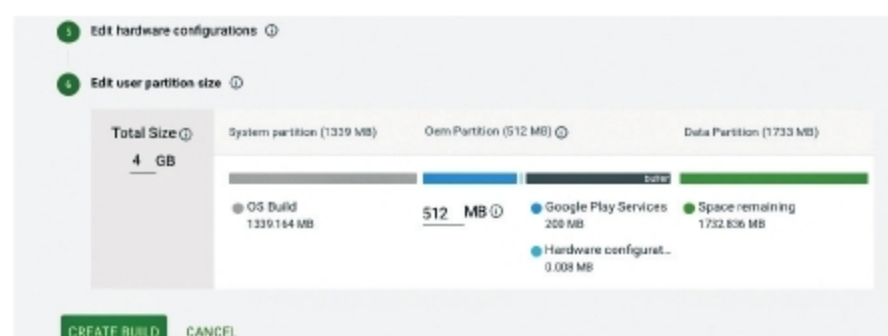


Figure 4: The hardware configuration and partition menu

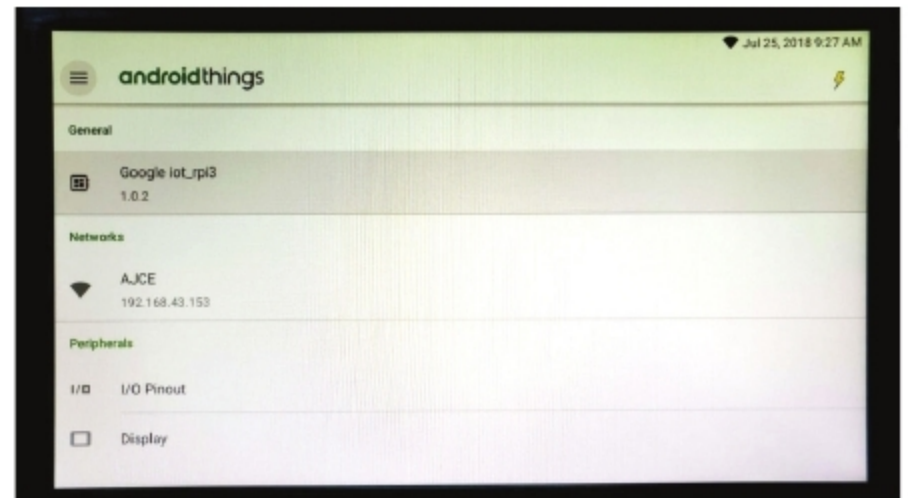


Figure 5: Android Things' home page. Connected to Wi-Fi.

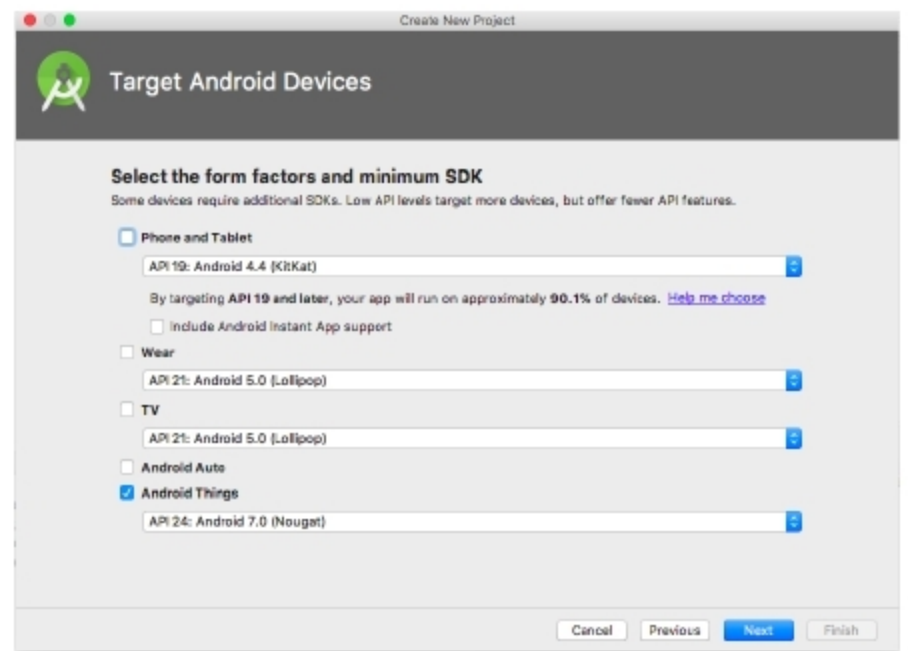


Figure 6: Select Android Things as the target



Figure 7: Raspberry Pi pinout

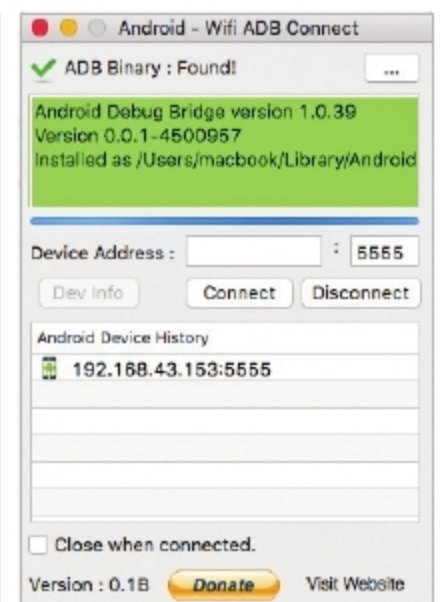


Figure 8: Android ADB connect

Open Android Studio and create a new project. Select *Android Things* as the target output device (Figure 6).

Select *Android Things Empty Activity* as the layout for this example.

We use Kotlin for development; so create a Kotlin class for storing default values. Declare the following object. Here we use BCM13. It can be changed according to the board selected. See Figure 7 for the Raspberry Pi 3 board structure.

```
val gpioForLED = when (Build.DEVICE) {
```